

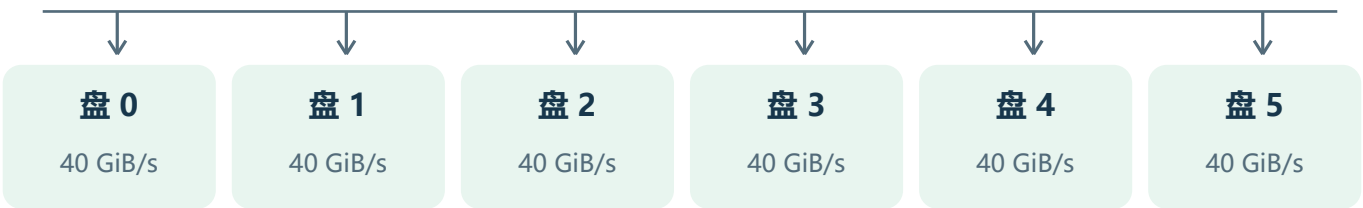
同一批活，为什么换个顺序就慢了？

先把问题想成一间工厂，再回到真实的计算卡和存储盘。

把 **NPU** 想成“做工的师傅”，把 **SSU** 想成“发材料的仓库”。师傅拿到数据才能计算；材料没来，即使还有很多任务，也只能等。这次用程序模拟设备运行，不是真机压测。



32 张 NPU：负责计算



6 块盘一起给 32 张卡供数。每块盘最多提供 40 GiB/s，合计 240 GiB/s；每张卡的接收链路上限是 50 GiB/s。GiB/s 表示每秒传数据的速度。下文的 SSU、SSD 都指实验里的存储盘。

这次只问一个问题

硬件相同、每卡的任务集合相同，只改变每张卡处理任务的先后顺序，**Baseline 会不会多花时间等数据？**

Baseline 是本次被比较的原始策略，第 3 页会画出它的排队方式。找到的一个例子从 92.92% 降到 83.60%；下面一步步解释数字。“超过 10%” 只在一个随机起点上成立。

先看一张卡：它怎样做一个请求？

“请求”就是一个任务；这次仿真把一个任务分成连续的 8 层。

每一层都要读数据、再计算。关键优化叫 **预取**：算第 1 层时，就提前读第 2 层的数据。这样，读数据和计算可以重叠。



它能否顺利衔接，取决于：**下一层的数据，有没有在当前层算完前送到。**

情况一：数据先到，计算接得上



情况二：数据晚到，计算卡停下来等



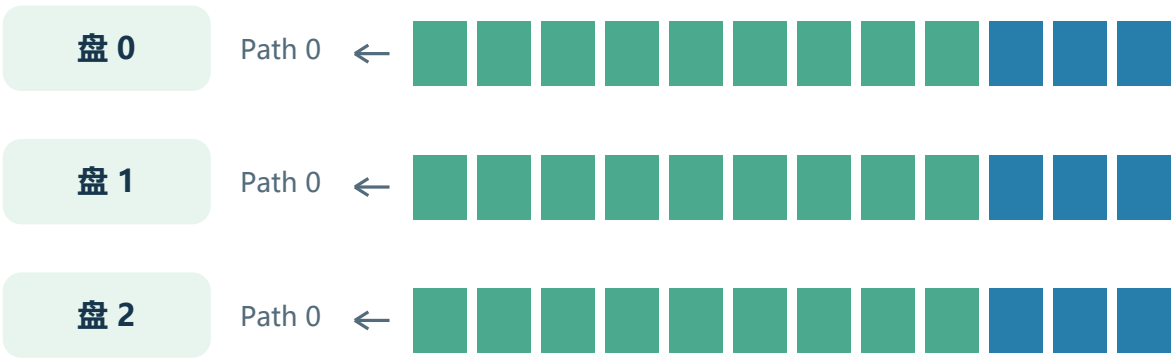
示意图用的是方便理解的数字，不是某一条真实轨迹。
利用率只把“正在计算”算进去；等数据的时间不算。

最后一层还有一个动作：它会预取“下一条请求”的第一层。这项操作在本次实验中一直保留，第 4 页会解释如何记带宽。

Baseline 怎样排队？

FIFO 读作“先来先服务”：同一条队伍里，先到的数据块先被处理。

Baseline 把每块盘上的所有读取都送到 **Path 0**。Path 就是排队通道；这里的 I/O 指读数据并传给卡。
六块盘各有自己的队伍，并不是六块盘共用一条总队伍。



上图仅画 3 块盘，另外 3 块同理。绿色、蓝色都是**同样大小的数据块**；每块 176 KiB。长读取占很多块，短读取只占几块。图中排列只是帮助理解的示意。

本实验的一层读取	数据块总数	分到六块盘以后
短 A / B / C	7 / 6 / 5 块	每盘通常 0-2 块
长请求	1530 块	每盘 255 块

可能挡住短请求的是前面的大量读取，不是别人的长计算。
别人在自己的卡上算得久，不会占住我的卡；但它先排进公共存储盘的材料，可能让我等。

长读取的所有块并不保证连续排在前面：多张卡会交错提交。因此“长短混合”本身不足以推出低利用率，还要看读取量、提交时刻和队列顺序。

“带宽没超”，为什么还可能排队？

把“任务按进度需要多快供数”和“此刻有多少材料排着队”分开。

这次约束的是一张“用料速度表”

单张卡的名义需求 = 每层读取量 ÷ 每层计算时间
对每块盘，把 32 张卡当前任务在这块盘上的需求加起来；必须始终小于 40 GiB/s。

例如：一项任务每层用 1 MiB 数据，计算 1 ms。按进度折算，就是每秒用 1000 MiB，约 0.98 GiB/s。它不意味着系统会把读取均匀摊在每一微秒。

用料速度不高，也可以集中来领料

设仓库每秒能发 100 箱。10 位师傅按手头任务折算，每人每秒需要 8 箱，合计 80 箱，没有超过能力。但如果大家同一刻来领材料，仍会出现队伍。能否影响开工，还要看每个人能等多久。

本实验检查的事	没有由这个条件自动保证的事
每个实际时刻，当前任务需求之和低于盘容量	每次读取都立即完成；没有瞬间积压；每层都赶得上预取期限

具体怎么查：每当有卡开始或结束处理任务，就重新计算六块盘的名义需求；两个这样的时刻之间，当前任务组合没有变化。所以检查的是全部变化时刻，不是每隔一段时间抽查。

你提出的“下一请求首层预取”，这样处理

短请求最后一层计算时，仍记当前短请求原有的名义需求，**不再额外加一份长请求首层的需求**。下一条请求正式接纳，也就是从待办队列取出并开始处理后，再切换成它的需求。

首层数据仍真实读取。**只有让计算卡停下来的等待，才计入利用率损失；已与计算重叠的读取时间不重复扣除**。这部分 I/O 没有被删掉。

单位提示：1 秒 = 1000 ms；1 GiB = 1024 MiB。GiB/s 表示每秒可传多少数据。

实际给每张卡准备了哪些任务？

先固定任务清单，再谈顺序。每张卡都有三种短任务和一种长任务。

名称	每层计算	每层读取	每卡条数
短 A	0.528 ms	1.203 MiB	200
短 B	0.697 ms	1.031 MiB	200
短 C	0.899 ms	0.859 MiB	200
长	25.612 ms	262.969 MiB	8

每张卡 608 条；32 张卡共 19456 条。
每条都计算 8 层。例如，长请求只算不等的总时间为 $25.612 \times 8 \approx 204.9$ ms。

为什么短任务这么多？

因为它们每条算得很快。如果只放几条短任务，哪怕它们很难受，整机按时间平均时也可能看不出来。这个配比让短任务占完整输入纯计算时间的约 **67.47%**。

短计算，不等于带宽需求一定大

还要看读取量。短 A 每层只读约 1.2 MiB，长请求要读约 263 MiB。本例短 A/B/C 的名义需求约为 2.23/1.44/0.93 GiB/s，长请求约为 10.03 GiB/s。

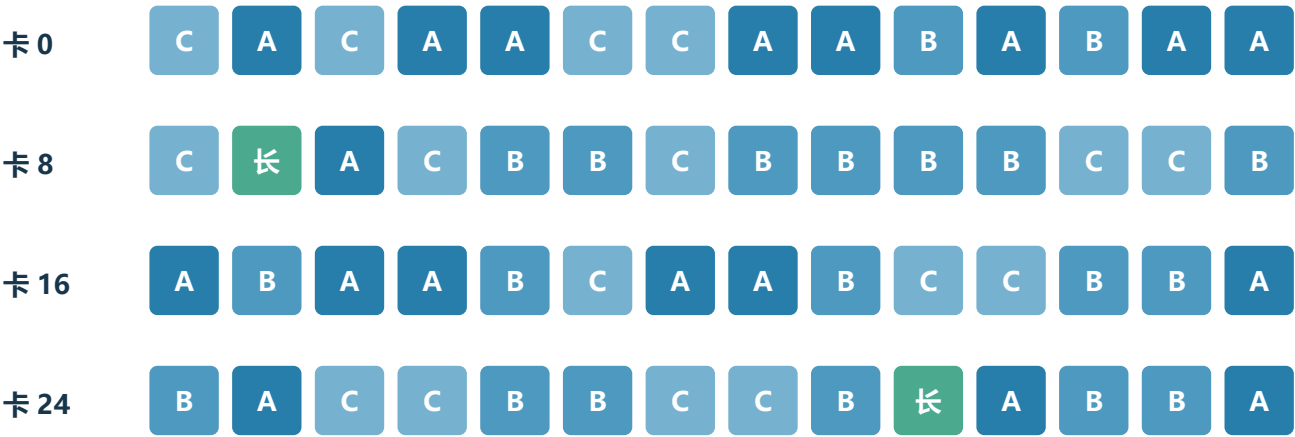
来源要说清：这些依据项目的 data 参数文件构造。短任务计算时间包含外推，即用已有较大输入的数据估算更小输入；长任务包含插值，即用两个已知点估算中间点。**它不是从真实业务采集的一整段请求记录。**

第一遍：每张卡各自洗牌

先测独立随机顺序下的 Baseline，作为比较起点。

像洗扑克牌一样：每张卡保留自己的 608 张任务卡片，各自打乱。谁先谁后由随机数决定；不会把一张卡的任务搬到另一张卡。

下面是真实输入的前 14 条



A/B/C 都是不同的短任务；绿色“长”是长任务。这里只展示开头，后面还有任务。例如卡 0 的第一条长任务在第 58 位，因此不能只看开头就说它没有长任务。

保持相同	允许改变
每卡任务条数、计算时间、读取量、数据在哪块盘、硬件和预取规则	第一遍是随机顺序；第二遍只改变每卡任务顺序

所有任务在仿真开始时进入各自 NPU 的待办队列，保证持续有活。**这不等于把所有未来层的数据同时交给 SSD。**读取仍随计算逐层触发。

这一遍，种子 7 的 Baseline 利用率为 **92.92%**。“种子”是可复现的随机数起点，后面还会换两个种子检查结果。

第二遍：清单不变，安排一种更容易撞车的顺序

不是让某些卡永远跑长任务；每张卡仍然处理长、短两类。

先拼一个 76 条任务的小段



长 1 条 + (短 A、短 B、短 C) 重复 25 遍
这个小段重复 8 次，正好仍是：长 8 条，短 A/B/C 各 200 条。

再把 32 张卡分成四组，从列表不同位置开始

卡号	卡数	把列表开头多少条搬到末尾
0-7	8	0 条
8-15	8	1 条
16-23	8	21 条
24-31	8	48 条

“搬到末尾”不是删除。例如 [A, B, C, D] 搬走开头 1 条，变成 [B, C, D, A]。真实输入对整张 608 条列表做同样操作。

这些数字怎么选：一个小段只算不等约需 630 ms，先瞄准它的 0、1/4、1/2、3/4 进度。因为不能把请求切成两半，就选择附近的完整请求边界，得到 0、1、21、48 条。

为什么这样做？ 让一组卡进入长任务时，另外几组更可能正在跑短任务；然后不同组轮流制造较大的读取。这里只安排输入列表，不在运行时强行暂停或同步卡。

同组八张卡用相同的任务类型顺序，是人为制造的相关性。它是寻找弱点的压力例子，不能代表各卡始终独立随机的现实流量。

为什么这套排列可能让短任务等得更久？

抓住一个时间差：短任务能等多久，前方的数据又要读多久。

短任务给预取留下的时间非常少

短 A/B/C 算完一层只需 **0.528–0.899 ms**。如果下一层数据排队加传输超过这个时间，当前层做完后就会停下来。

一批大读取可能占据几毫秒的盘服务

假设八张卡的一层长读取集中排在短读取前面：

每盘数据量 = $8 \times 262.969 \div 6 \approx 350.625 \text{ MiB}$

盘服务时间 = $350.625 \text{ MiB} \div 40 \text{ GiB/s} \approx \mathbf{8.56 \text{ ms}}$

8.56 ms 明显大于短任务的一层计算时间。长任务自己每层能算 25.612 ms，反而有更长时间隐藏读取。上面是说明机制的假设，不是说真实队列每次都恰好堆着这八份完整读取。

自然错开会不会把问题修好？

会有帮助。随机顺序下，不同卡的长读取比较分散。卡一旦等待，后续时间也会发生偏移；这种偏移可能减少下一次碰撞，也可能让它撞上另一组。结果必须看实际运行。

这次用精确重复的小段和四组不同起点，尝试让干扰反复出现，而不是只在启动时堵一次。但它没有保证一直同步，也没有保证每个种子都下降 10%。

这次构造刻意组合了：**大读取突发、短预取时间、足够多短任务的计算份额，以及不利的跨卡时序**。它们是寻找弱点的思路，并非所有低利用率问题都必须满足的条件。

怎样测量？这些百分数到底是什么意思？

先让系统跑起来，这叫暖机。固定观察第 2 秒到第 4 秒，不挑最差的两秒。

开始观察前，每卡早已完成至少 4 条任务。主窗口长 2 秒；一张卡占用 1 秒就是 1 卡·秒，因此 32 张卡合起来有 $32 \times 2 = 64$ 卡·秒 可用。

NPU 平均利用率 = 所有卡真正计算的时间 ÷ 64 卡·秒

卡有任务却在等数据，仍算“活跃”，但这段时间不算计算利用率。



把总卡时间想成 100 份：原来约 93 份在计算，现在约 84 份在计算，其余在等数据。蓝色与绿色合计是计算，橙色是等待。下面用更多小数位计算，避免舍入误差。

比较方式	怎么计算	结果
百分点差	92.922866 – 83.596252	约 9.33 个百分点
相对降幅	9.326614 ÷ 92.922866	约 10.04%

换随机种子再试，不能只留最好看的结果

种子	随机 → 重排	相对下降
7	92.92% → 83.60%	10.04%
19	92.04% → 85.47%	7.14%
43	91.30% → 85.51%	6.34%

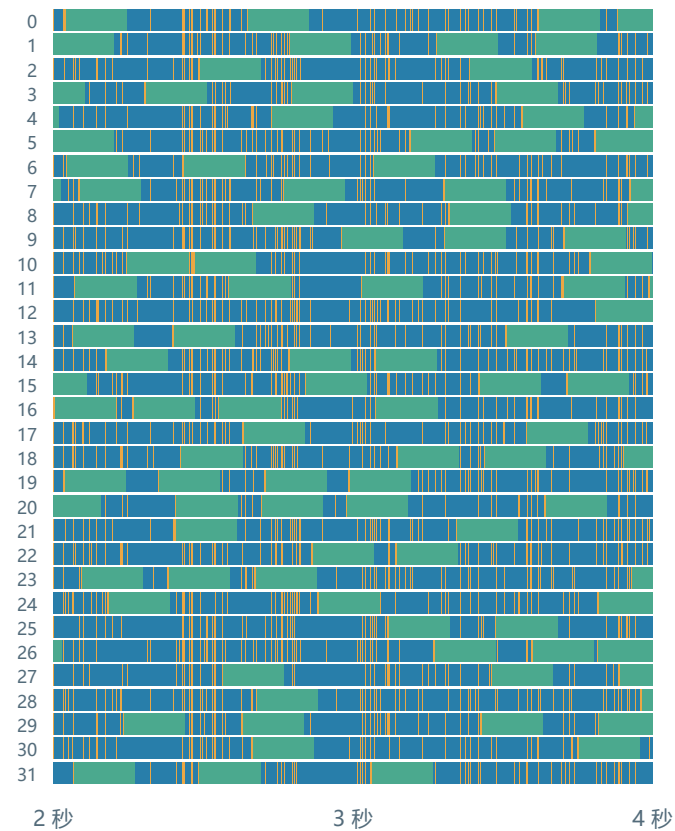
每对比较使用同一个提交种子；换一行时，随机排序和提交时序的随机起点都变了。相同的坏顺序模板也会跑出不同轨迹。**所以超过 10% 目前只是一个刚过线的例子。**

真实运行长什么样？

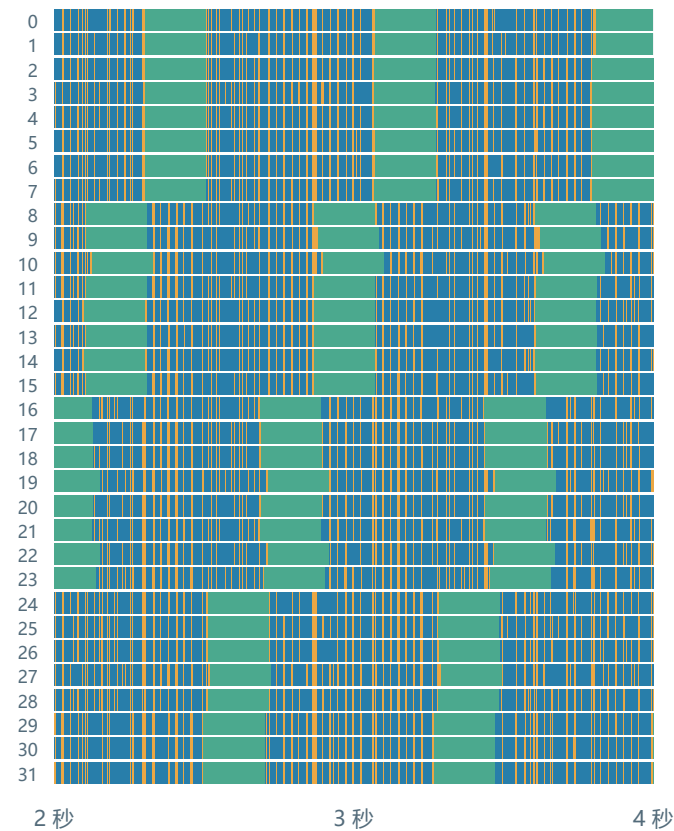
一行是一张卡，横向是第 2-4 秒；橙色越多，等数据越久。

蓝色：短任务正在计算。绿色：长任务正在计算。橙色：已经接纳了任务，但数据没到。下面保留全部 32 张卡，帮助检查是否每卡都跑过长、短任务。

随机：92.92%



重排：83.60%



类别利用率只看处理该类任务的时间：计算 ÷ (计算 + 等数据)。短类、长类各算各的，不能相加，也不是各自除以 64 卡·秒。

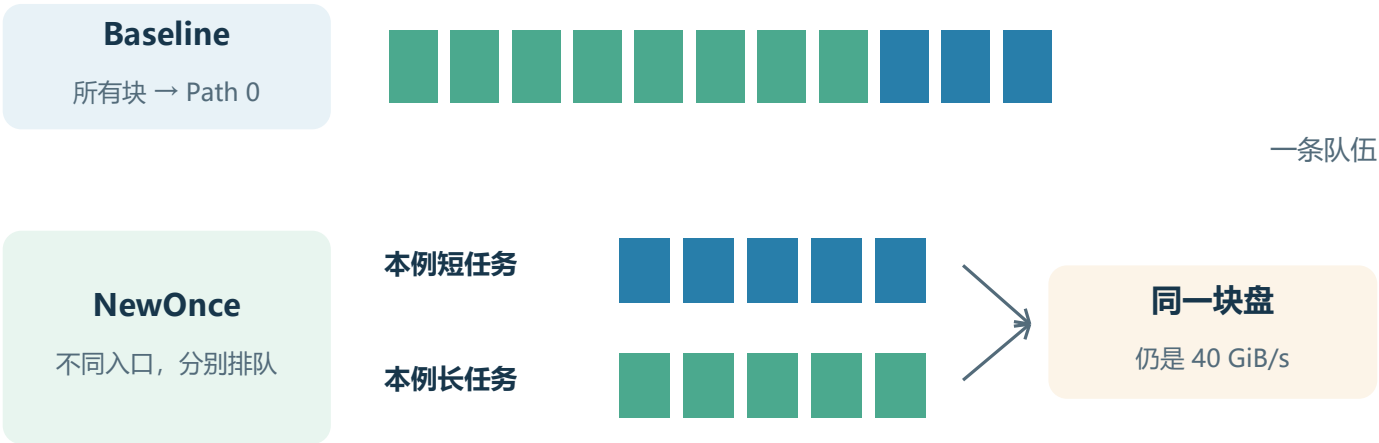
检查结果	随机	重排
短类利用率	90.78%	78.21%
长类利用率	97.67%	97.46%
全程逐盘名义峰值	30.38 GiB/s	34.14 GiB/s

两种输入都满足：每卡整个窗口有任务、每卡都有长短计算、全程每盘名义需求低于 40。
新增等待约 88.83% 出现在任务内部后续层，并非主要来自下一请求首层预取。

NewOnce 改了哪一环？

它在数据块发出前选择队列，再由盘在不同队列间分配服务；NPU 的任务清单不重排。

继续用仓库类比：Baseline 让所有人站同一队。NewOnce 使用仓库已有的多个排队入口，让不同类型的数据分开排；发料员再按配置在各队伍之间选择下一箱。



每盘的逻辑队列	条数	竞争时的基础份额
本例短 A/B/C 可用	96 条	合计 20 GiB/s
本例长请求可用	32 条	合计 6 GiB/s
另外两种代码类别	128 条	合计 14 GiB/s

每盘总共 **256 条逻辑 FIFO 队列**，物理上仍只有一块盘。基础份额用于竞争时的服务分配，空闲份额可以借给其他队列，不是固定带宽上限。

盘每次仍只读取一个 176 KiB 数据块。读完当前块后，哪怕长队伍还没清空，也可以按服务份额轮到短队伍。**多队列改变服务先后，不增加总带宽。** Baseline 的 Path 0 也能借用空闲份额，并没有被锁死在极低带宽。

本例短任务的代码类别为 SS，长任务为 LL；代码分类来自输入参数，不是运行时测出“谁算得快”后临时分组。每条队列内部仍然先来先服务。

NewOnce 每次怎样为数据块选路？

“Once” 表示对一组数据集中规划一次，不表示整层只能用一条路径。

1 确定这批数据属于哪块盘

数据放在哪块盘早已固定。对“一个请求的一层、在一块盘上的所有块”，集中做一次路径规划。

2 查看大家共用的预约账本

看每条路径上，还有多少块已经被安排、但尚未确认到达卡上内存。这包括预约了却还没实际发出的块。

3 给第一个块挑一个预计更快的入口

只在该类任务允许使用的路径中比较。看前面有多少未完成数据，再按前页的基础份额和可借的空闲份额，估计这条路径能分到多少带宽。

4 临时记上一笔，再安排下一个块

前一个块刚选过的路径，已经多了一份工作；后一个块不能把它当成还没被选过。不同块可以走不同路径。

5 把整批预约登记到共享账本

登记完成后，下一个客户端才继续规划。之后照原有节奏实际提交；等数据到达 NPU 内存，收到确认，再扣掉对应预约。

选路时的直觉：

预计时间 $\approx$ (队里未确认的数据量 + 当前块大小) $\div$ 预计分到的带宽。

选择预计时间较小的路径，而不是只数谁的队伍最短。

这是选路估计，不是完成时间保证。当前 NewOnce 也不是直接按“谁的截止时间最近”排序；它通过分类路径池、服务份额和预约协调改变 I/O 次序。

为什么需要一本“所有卡都看得到”的账？

设备采样可能过时，但客户端自己刚安排了什么，可以立即共享。

先区分两个名字：**Once** 使用最近一次设备采样做规划；**NewOnce** 使用全体客户端的预约与完成确认账本重建选路信息。

信息来源	多久更新	内容
设备采样	0、5、10... ms	那次采样时盘上各路径的状态
NewOnce 共享账本	预约后立即加；完成确认后立即减	所有客户端已计划、未确认的数据块

一个已用当前选路函数核对的小例子

同一块盘，旧采样显示路径全空。甲、乙两个短任务各准备 4 块数据；平局时的选择起点均设为 0，期间没有完成确认。下面是示意，不是实测轨迹。

发生的事	只看同一份旧采样	使用 NewOnce 共享账
甲选 4 个入口	0、32、64、96	0、32、64、96
甲选完后	乙仍看到旧的“全空”状态	立即登记甲的 4 个预约
乙再选 4 个入口	0、32、64、96	128、160、192、224

乙能看见甲刚预约的工作，就更容易避开同样的入口。**这些编号是同一块盘内部的逻辑队列号，不是额外的盘，也不是 NPU 编号。**

Once 也会记住本批前面几个块的选择。但下一个客户端看不到上一批尚未发出的预约；NewOnce 会把这些预约共享出来。

NewOnce 的选路计算使用共享账本，**不是每隔 5 ms 才更新一次选择，也不是实时偷看 SSD 内部队列**。账本直到数据到达 NPU 内存才扣除，因此可能比盘上真实排队量更保守。

必要条件：所有相关 I/O 都受这套客户端管理，预约更新能协调一致。若还有账本不知道的外部流量，就不能直接沿用这里的效果结论。

换成 NewOnce 后，发生了什么？

用同一个较坏输入做对照：只换 I/O 路径策略，任务清单与顺序都保留。



指标及统计范围	Baseline	NewOnce
第 2–4 秒：短类利用率	78.21%	100.00%
第 2–4 秒：内部后续层等数据停工时间	8.881 卡·秒	0 卡·秒
完整运行：逐盘名义峰值	34.14 GiB/s	32.94 GiB/s

在第 2–4 秒窗口内，短任务没有因数据迟到而停下来等，整机利用率达到 **99.14%**。两种策略都满足本例的名义欠载和暖机要求。完整这批请求完成时间也从约 **6.053 秒**降到 **5.104 秒**。

读完后，应能讲清这四件事

- ① **怎么做**：每卡固定 608 条任务，先独立洗牌，再改成四组不同起点的长短段；固定看第 2–4 秒。
- ② **为什么可能慢**：短计算给预取的时间少，大读取在公共盘上的排队可能使它错过开工时间。
- ③ **NewOnce 怎么帮**：为数据块选不同的合适入口，用共享预约账减少客户端之间的重复拥挤，再由盘在多队列间分配服务。
- ④ **没有证明什么**：不是所有混合输入都差 10%，也不是多开队列必然有效。

结果边界：短任务参数经过构造；组内顺序有意相关；NewOnce 同时改变选路、分类隔离和服务次序，不能把收益全归给路径数量。模型的控制通信/计算延迟设为 0。长类也有小幅让步：完整批次长类利用率约 97.43% → 97.15%。

查证位置：同实验目录的 report.md 是完整报告；all_results.md 保留全部 39 次运行，analysis.json 是逐事件核算。NewOnce 源码见 shared_path_new_once.py、shared_path_sim_adapter.py、policy_logic.py；本 PDF 的生成脚本与来源哈希保存在 tutorial 目录。